BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY CHƯƠNG TRÌNH

LikTok – Ứng dụng chia sẻ video ngắn

Đề tài 1: Video Sharing App

Môn	: Lập trình cho thiết bị di động
Giảng viên hướng dẫn	: Nguyễn Minh Hải
Lớp học phần	: DHKTPM19B – 420300143202
Nhóm thực hiện	: Nhóm 1
Thành viên	: Phạm Quốc Khánh (nhóm trưởng) – 23641111
	Nguyễn Anh Long – 23633881
	Huỳnh Hoài Thanh – 23633801
Mã nguồn	: github.com/khanh2925/video-sharing-app

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

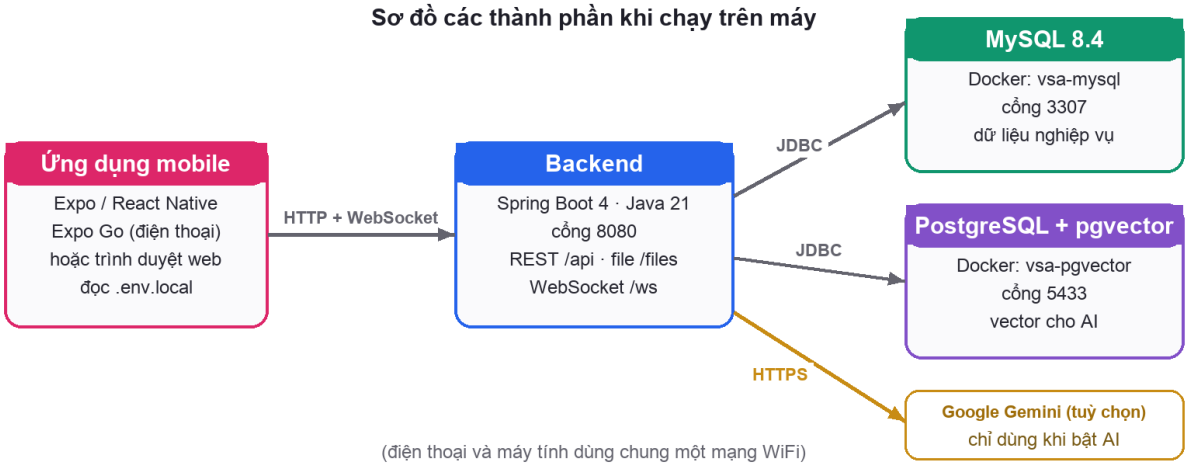
1. Giới thiệu 3
2. Cách nhanh nhất: dùng bản đã triển khai sẵn 3
3. Chuẩn bị 4
3.1. Phần mềm cần cài (mỗi máy một lần) 4
3.2. Yêu cầu về mạng 5
4. Bước 1: Lấy mã nguồn 5
5. Bước 2: Chạy cơ sở dữ liệu bằng Docker 5
6. Bước 3: Chạy backend 6
6.1. Cách A: dùng IntelliJ IDEA 6
6.2. Cách B: dùng dòng lệnh (không cần IntelliJ) 6
6.3. Kiểm tra backend 7
6.4. (Tuỳ chọn) Bật các chức năng AI 7
7. Bước 4: Chạy ứng dụng mobile 8
7.1. Cài thư viện và khai báo địa chỉ backend 8
7.2. Khởi động Expo 8
7.3. Kiểm tra nhanh các chức năng 10
8. Chạy unit test 10
9. Lệnh thường dùng 11
10. Lỗi thường gặp và cách xử lý 11
11. Triển khai lên máy chủ (tham khảo) 12

1. Giới thiệu

Tài liệu này hướng dẫn cách lấy mã nguồn LikTok từ GitHub, cài các phần mềm cần thiết và chạy toàn bộ hệ thống trên một máy tính cá nhân, gồm: cơ sở dữ liệu, backend và ứng dụng mobile. Mọi lệnh và hình minh hoạ trong tài liệu được chụp lại từ lần chạy thử thật trên nhánh **main** của repo.

Hệ thống gồm bốn phần chạy cùng lúc:

Thành phần	Công nghệ	Thư mục	Cổng
Ứng dụng mobile	Expo SDK 58, React Native, TypeScript	`mobile/`	8081 (Metro)
Backend (API)	Spring Boot 4, Java 21, Maven	`backend/`	8080
CSDL nghiệp vụ	MySQL 8.4 (Docker)	`docker-compose.yml`	3307
Kho vector cho AI	PostgreSQL 17 + pgvector (Docker)	`docker-compose.yml`	5433



Hình 1. *Các thành phần của LikTok khi chạy trên máy cá nhân*

> **Lưu ý:** phần AI (trợ lý LikTok, tìm kiếm theo ý nghĩa, mục "Dành cho bạn" dùng vector) là **tuỳ chọn**. Không bật AI thì các chức năng còn lại vẫn chạy đầy đủ; nút trợ lý sẽ báo "Trợ lý AI đang tắt".

2. Cách nhanh nhất: dùng bản đã triển khai sẵn

Nhóm đã triển khai bản mới nhất của nhánh main lên máy chủ (Oracle Cloud VPS, có HTTPS). Nếu chỉ cần xem và dùng thử, thầy/cô không phải cài gì, chỉ cần mở trình duyệt vào địa chỉ:

```
https://liktok.129-221-164-93.sslip.io
```

Có thể dùng các tài khoản mẫu bên dưới (do chương trình tự tạo khi khởi động lần đầu) hoặc tự đăng ký tài khoản mới.

Tài khoản	Mật khẩu	Vai trò
`admin`	`123456`	Quản trị viên (kiểm duyệt video, quản lý người dùng, cấu hình hệ thống)
`lan`, `minh`, `hoa`, `an`	`123456`	Người dùng thường, đã có sẵn video, bình luận, lượt theo dõi

Muốn chạy bản mobile trên điện thoại nhưng vẫn dùng máy chủ này, làm theo mục 7 và đặt `EXPO_PUBLIC_API_URL` bằng địa chỉ trên thay cho IP máy tính. Khi đó không cần chạy Docker và backend trên máy.

3. Chuẩn bị

3.1. Phần mềm cần cài (mỗi máy một lần)

Phần mềm	Phiên bản	Dùng để
Git	bất kỳ	Lấy mã nguồn từ GitHub
Docker Desktop	bản mới	Chạy MySQL và PostgreSQL + pgvector, không phải cài CSDL thủ công
JDK	21	Chạy backend Spring Boot
IntelliJ IDEA (tuỳ chọn)	Community hoặc Ultimate	Mở và chạy backend bằng giao diện. Không có thì chạy bằng lệnh Maven Wrapper đi kèm repo
Node.js	20 trở lên (nhóm dùng 22)	Cài thư viện và chạy ứng dụng Expo
Expo Go (trên điện thoại)	hỗ trợ SDK 58	Mở ứng dụng trên iPhone/Android bằng cách quét mã QR. Có thể thay bằng iOS Simulator hoặc Android Emulator

Kiểm tra nhanh các phần mềm đã cài đúng chưa:

```
git --version
docker --version
java -version        # phải là 21
node -v              # v20 trở lên
```

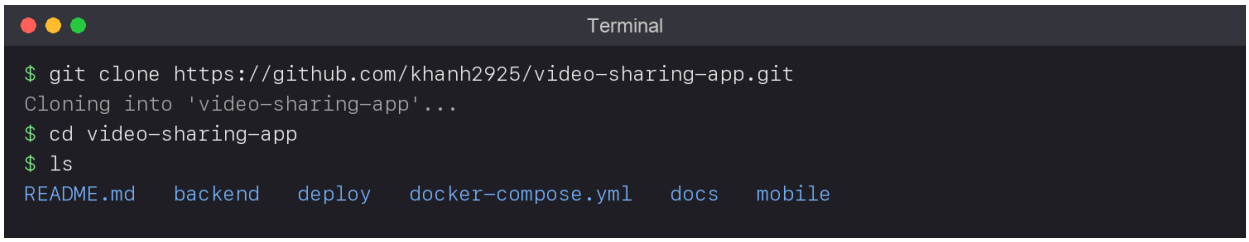
3.2. Yêu cầu về mạng

- Điện thoại và máy tính chạy backend phải **cùng một mạng WiFi**, vì điện thoại gọi backend qua IP nội bộ của máy tính.
- Nếu dùng iOS Simulator hoặc chạy bản web ngay trên máy tính thì không cần điều kiện này.

4. Bước 1: Lấy mã nguồn

Repo GitHub của nhóm đang ở chế độ riêng tư. Tài khoản GitHub dùng để tải cần được nhóm cấp quyền xem (Collaborator) trước. Sau đó mở Terminal (Windows dùng PowerShell hoặc Git Bash) và chạy:

```
git clone https://github.com/khanh2925/video-sharing-app.git
cd video-sharing-app
```



Hình 2. *Tải mã nguồn và các thư mục chính của repo*

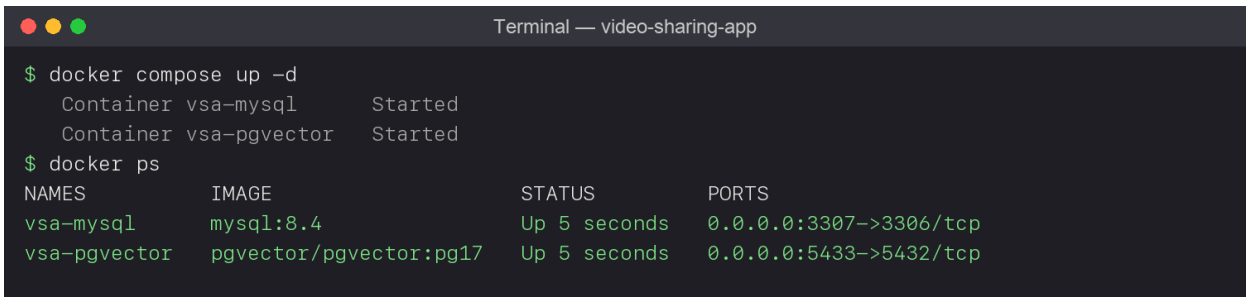
Thư mục / tệp	Nội dung
`backend/`	Mã nguồn Spring Boot, cấu hình `application.yaml`, video mẫu (`seed-media/`), unit test
`mobile/`	Mã nguồn ứng dụng Expo (React Native + TypeScript)
`docker-compose.yml`	Khai báo hai container CSDL dùng khi chạy trên máy
`deploy/`	Kịch bản triển khai lên VPS (Docker Compose, Caddy HTTPS)
`docs/`	Tài liệu SRS/SDS, báo cáo, nghiệp vụ từng chức năng

5. Bước 2: Chạy cơ sở dữ liệu bằng Docker

Mở **Docker Desktop** và đợi biểu tượng Docker báo đang chạy. Tại thư mục gốc `video-sharing-app`, chạy:

```
docker compose up -d
docker ps
```

Thấy hai container `vsa-mysql` và `vsa-pgvector` ở trạng thái **Up** là được. Lần đầu Docker phải tải image nên mất vài phút.



Hình 3. *Hai container cơ sở dữ liệu đã chạy*

Thông số	MySQL	PostgreSQL + pgvector
Máy chủ : cổng	`localhost:3307`	`localhost:5433`
Tên CSDL	`video_sharing`	`vsa_vectors`
Tài khoản / mật khẩu	`root` / `root`	`vsa` / `vsa`

> **Ghi chú:** không cần tạo bảng bằng tay. Khi backend khởi động, Hibernate tự tạo bảng theo các lớp Entity, sau đó lớp `DataSeeder` tự nạp dữ liệu mẫu.

6. Bước 3: Chạy backend

6.1. Cách A: dùng IntelliJ IDEA

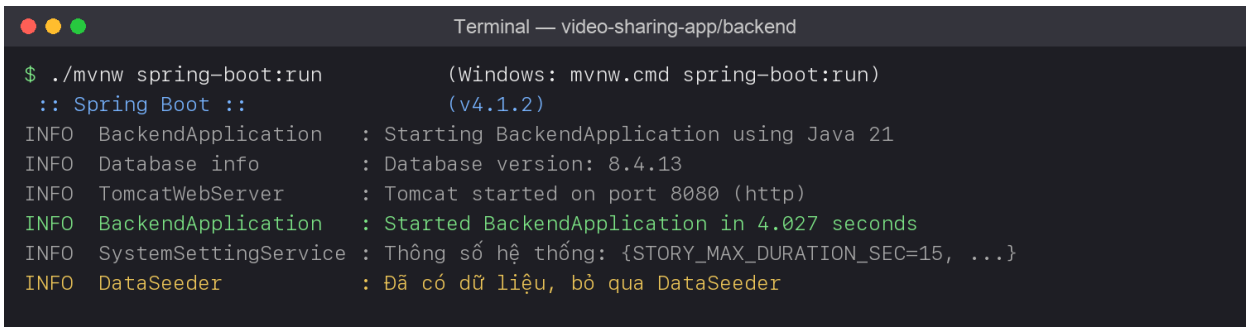
1. IntelliJ → **Open** → chọn thư mục `backend` (không chọn thư mục gốc).
2. Đợi IntelliJ tải xong thư viện Maven (thanh tiến trình góc dưới).
3. Kiểm tra **File → Project Structure → SDK** là Java 21.
4. Mở `src/main/java/com/vsa/backend/BackendApplication.java`, bấm nút **Run** (hình tam giác màu xanh) cạnh hàm `main`.

6.2. Cách B: dùng dòng lệnh (không cần IntelliJ)

Repo có sẵn Maven Wrapper nên không phải cài Maven:

```
cd backend
./mvnw spring-boot:run            # macOS / Linux
mvnw.cmd spring-boot:run          # Windows
```

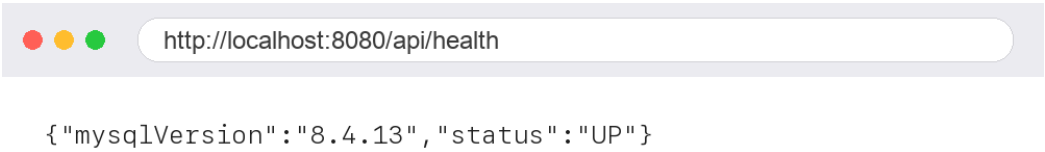
Khi thấy dòng **Started BackendApplication** và **Tomcat started on port 8080** là backend đã chạy.



Hình 4. *Nhật ký khởi động backend (lần chạy thứ hai nên DataSeeder bỏ qua)*

6.3. Kiểm tra backend

Mở trình duyệt vào `http://localhost:8080/api/health`. Kết quả có `"status":"UP"` và phiên bản MySQL nghĩa là backend đã kết nối được cơ sở dữ liệu:



Hình 5. *Backend trả về trạng thái UP*

Lần chạy đầu, `DataSeeder` tự nạp dữ liệu mẫu: các tài khoản ở mục 2, video mẫu kèm ảnh bìa, chủ đề, nhạc nền, bình luận và một phiên livestream giả lập. Có thể thử mở `http://localhost:8080/files/videos/video01.mp4` để xem một video mẫu.

6.4. (Tuỳ chọn) Bật các chức năng AI

AI mặc định tắt. Muốn bật trên máy cá nhân:

1. Đảm bảo container `vsa-pgvector` đang chạy (bước 2).
2. Vào thư mục `backend/src/main/resources`, chép tệp mẫu `ai-local.example.yaml` thành `ai-local.yaml`. Tệp này đã nằm trong `.gitignore` nên không bị đẩy lên GitHub.
3. Chạy lại backend. Trong nhật ký phải thấy profile **"ai"** đang hoạt động.
4. Đăng nhập tài khoản `admin` rồi gọi `POST /api/admin/ai/reindex` một lần để tạo vector cho các video có sẵn. Video đăng sau đó sẽ tự tạo vector.

```
cd backend/src/main/resources
cp ai-local.example.yaml ai-local.yaml        # macOS / Linux
copy ai-local.example.yaml ai-local.yaml      # Windows
```

Khoá API của Gemini và của máy chủ chat đã được nhóm cấu hình sẵn trong `application-ai.yaml`. Muốn dùng khoá khác thì đặt biến môi trường cùng tên trước khi chạy backend (biến môi trường được ưu tiên hơn giá trị trong tệp):

Biến môi trường	Ý nghĩa
`GEMINI_API_KEY`	Khoá Google Gemini, dùng để tạo vector (bắt buộc khi bật AI)
`AI_CHAT_PROVIDER`	Bên trả lời chatbot: `openai` (máy chủ tương thích OpenAI, mặc định) hoặc `google-genai` (Gemini)
`OPENAI_API_KEY`, `OPENAI_BASE_URL`, `OPENAI_CHAT_MODEL`	Khoá, địa chỉ (có `/v1` ở cuối) và tên model của máy chủ chat khi dùng `openai`

Muốn tắt AI thì xoá hoặc đổi tên tệp `ai-local.yaml` rồi chạy lại backend.

7. Bước 4: Chạy ứng dụng mobile

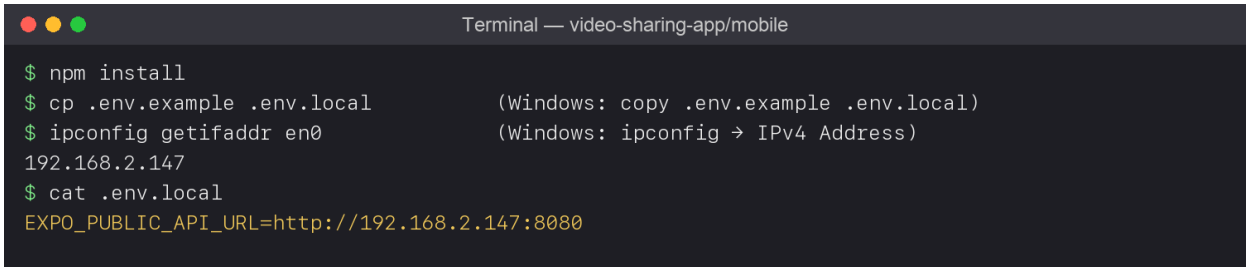
7.1. Cài thư viện và khai báo địa chỉ backend

Mở một cửa sổ Terminal mới (giữ nguyên cửa sổ đang chạy backend):

```
cd mobile
npm install
cp .env.example .env.local          # Windows: copy .env.example .env.local
```

Mở tệp `mobile/.env.local` và thay `<IP-may-ban>` bằng địa chỉ IP nội bộ của máy tính đang chạy backend:

- macOS: chạy `ipconfig getifaddr en0`.
- Windows: chạy `ipconfig`, lấy dòng **IPv4 Address** của card WiFi.



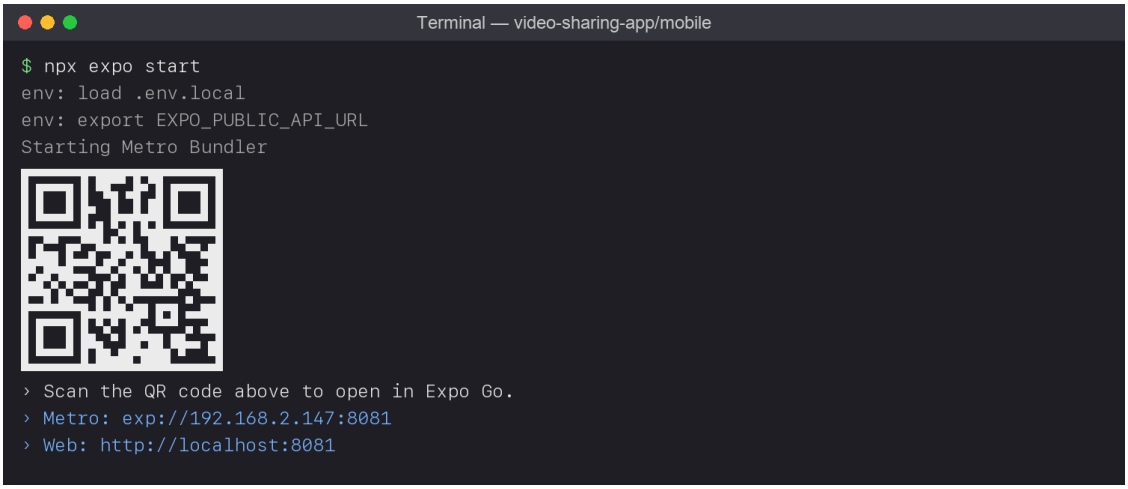
Hình 6. *Tạo tệp .env.local trỏ tới IP của máy tính*

> **Lưu ý:** không dùng `localhost` trong `.env.local` khi chạy trên điện thoại thật, vì với điện thoại "localhost" là chính nó chứ không phải máy tính. Muốn dùng máy chủ của nhóm thay cho backend trên máy thì ghi `EXPO_PUBLIC_API_URL=https://liktok.129-221-164-93.sslip.io`.

7.2. Khởi động Expo

```
npx expo start
```

Terminal in ra một mã QR và địa chỉ Metro:

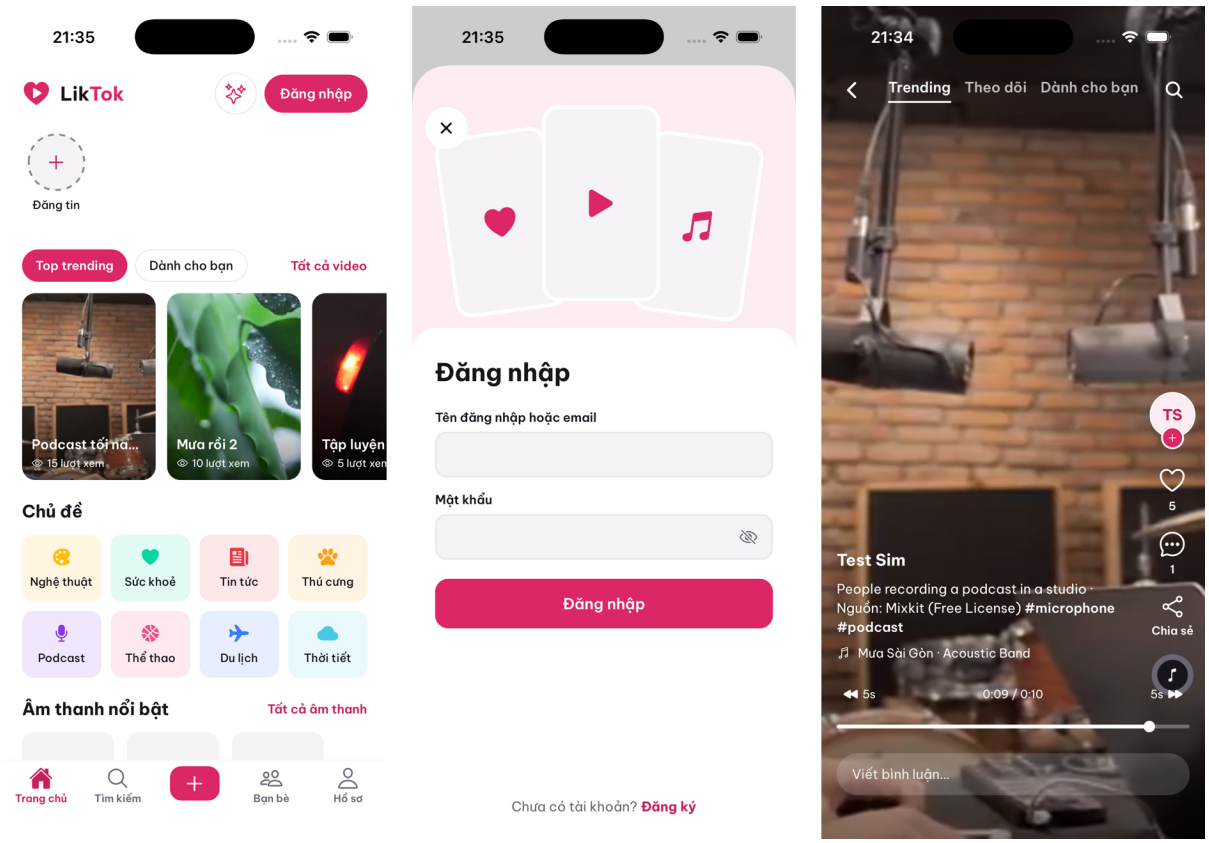


Hình 7. *Expo đã chạy và in mã QR*

Chọn một trong các cách mở ứng dụng:

Thiết bị	Cách mở
iPhone	Mở app **Camera**, quét mã QR, bấm thông báo để mở bằng Expo Go
Android	Mở app **Expo Go** → Scan QR code → quét mã QR
iOS Simulator (macOS)	Bấm phím `i` trong cửa sổ Terminal đang chạy Expo
Android Emulator	Mở emulator trước, rồi bấm phím `a`
Trình duyệt web	Bấm phím `w` (mở `http://localhost:8081`)

Lần đầu mở, Metro mất khoảng 30 giây đến 1 phút để đóng gói mã nguồn. Sau đó ứng dụng hiện **Trang chủ** ở chế độ khách: xem được video, tìm kiếm, xem hồ sơ; các thao tác như thích, bình luận, đăng video sẽ yêu cầu đăng nhập.



Hình 8. *Ứng dụng chạy trên iPhone 17 Pro Simulator với backend trên máy: Trang chủ, Đăng nhập, Xem video*

7.3. Kiểm tra nhanh các chức năng

1. Trang chủ hiện mục Top trending, Chủ đề, Âm thanh nổi bật với ảnh bìa video.
2. Bấm một video: video phát, có nút thích, bình luận, chia sẻ.
3. Bấm **Đăng nhập**, dùng `lan` / `123456`: vào được, tab Hồ sơ hiện thông tin của Lan.
4. Bấm nút + ở giữa thanh dưới: mở camera để quay hoặc chọn video từ máy và đăng.
5. Đăng nhập `admin` / `123456` để thử các chức năng quản trị.

8. Chạy unit test

Backend có bộ unit test JUnit 5 + Mockito. Cần container MySQL đang chạy (bước 2):

```
cd backend
./mvnw test                          # Windows: mvnw.cmd test
```

Mobile có bộ test chạy bằng trình kiểm thử có sẵn của Node.js:

```
cd mobile
npm test
# kết quả: tests 12, pass 12, fail 0
```

Hai bộ test này cũng tự chạy trên GitHub Actions mỗi khi mở Pull Request vào nhánh main (tệp `.github/workflows/test.yml`); chỉ khi test đạt thì mã mới được triển khai lên VPS.

9. Lệnh thường dùng

Việc cần làm	Lệnh (chạy ở thư mục gốc nếu không ghi khác)
Tắt các container CSDL	`docker compose down`
Xoá sạch dữ liệu, nạp lại dữ liệu mẫu từ đầu	`docker compose down -v` rồi `docker compose up -d` và chạy lại backend
Mở dòng lệnh MySQL	`docker exec -it vsa-mysql mysql -uroot -proot video_sharing`
Tắt Expo hoặc backend	Bấm `Ctrl + C` trong cửa sổ Terminal tương ứng
Xoá bộ nhớ đệm Metro khi app lỗi lạ	`npx expo start -c` (trong thư mục `mobile`)
Lấy mã nguồn mới nhất	`git pull`

10. Lỗi thường gặp và cách xử lý

Hiện tượng	Nguyên nhân và cách xử lý
`Cannot connect to the Docker daemon`	Docker Desktop chưa mở. Mở Docker Desktop, đợi chạy xong rồi chạy lại lệnh.
`port 3307 is already allocated`	Cổng 3307 đang bị ứng dụng khác dùng. Tắt ứng dụng đó, hoặc đổi cổng trong `docker-compose.yml` và `backend/src/main/resources/application.yaml`.
Backend báo `Communications link failure`	MySQL chưa chạy hoặc chưa sẵn sàng. Chạy `docker compose up -d`, đợi 10–20 giây rồi chạy lại backend.
Backend báo lỗi phiên bản Java	Máy đang dùng JDK khác 21. Cài JDK 21 và chọn đúng SDK trong IntelliJ hoặc đặt `JAVA_HOME`.
Ứng dụng báo `Network request failed`	Sai IP trong `.env.local`, điện thoại khác mạng WiFi với máy tính, hoặc backend chưa chạy. Sửa lại rồi chạy lại `npx expo start`.
`JSON Parse error: Unexpected character <`	Địa chỉ trong `.env.local` trỏ nhầm sang máy hoặc ứng dụng khác. Kiểm tra lại IP và cổng 8080.
Đổi mạng WiFi xong app không tải được dữ liệu	IP của máy tính đã đổi. Lấy IP mới, sửa `.env.local`, chạy lại `npx expo start`.
Expo Go báo không hỗ trợ phiên bản SDK	Cập nhật Expo Go trên điện thoại lên bản mới nhất (dự án dùng SDK 58).

Hiện tượng	Nguyên nhân và cách xử lý
Trợ lý AI báo "Trợ lý AI đang tắt"	Chưa bật AI hoặc khoá hết lượt. Xem mục 6.4; có thể đổi `AI_CHAT_PROVIDER` sang bên còn lại rồi chạy lại backend.

11. Triển khai lên máy chủ (tham khảo)

Bản đang chạy tại https://liktok.129-221-164-93.sslip.io được triển khai bằng Docker Compose trên Oracle Cloud VPS, gồm các container: MySQL, PostgreSQL + pgvector, backend Spring Boot, Caddy (tự cấp chứng chỉ HTTPS, phục vụ cả bản web của ứng dụng) và Adminer (trang xem CSDL, mặc định tắt). Các bước chi tiết nằm trong tệp **deploy/README.md** của repo; tóm tắt:

1. Tạo VPS Ubuntu 24.04, mở cổng 80 và 443.
2. SSH vào VPS, clone repo, chạy `bash deploy/setup-vps.sh`. Script tự cài Docker, sinh mật khẩu CSDL và JWT ngẫu nhiên, build và chạy toàn bộ.
3. Các lần sau chỉ cần đẩy mã vào nhánh main: GitHub Actions chạy test, đạt thì tự SSH vào VPS chạy `deploy/update.sh` và kiểm tra lại `/api/health`.